ConstRSA: A MINIMAL CONSTANT-TIME RSA-512 DIGITAL SIGNATURE WITH STATISTICAL TIMING LEAK VERIFICATION ON STANDARD LAPTOPS

Research proposal

Submitted in partial fulfilment of the requirement for the degree of Bachelor of Science Honours in Information Technology

By:
Maleesha Rathnayake
2020/ICT/47

Department of Physical Science
Faculty of Applied Science
University of Vavuniya
Sri Lanka

December, 2025

1 Introduction

1.1 Background

RSA digital signatures play a crucial role in secure communication systems. However, many RSA implementations leak sensitive timing information that attackers can exploit to recover private keys. These timing variations occur when operations depend on secret data. To address this vulnerability, cryptographic functions must be implemented in constant time. This research proposes a minimal, fully constant-time RSA-512 digital signature implementation suitable for education, experimentation, and benchmarking on standard laptops.

1.2 Problem Definition

Most available RSA implementations are not constant-time, leaving systems vulnerable to timing attacks. Existing constant-time implementations are often large, complex, or designed for specialized hardware. There is a lack of small, accessible, and academically suitable constant-time RSA implementations.

1.3 Research Objectives

Primary Objective:
To design, implement, and statistically verify a minimal constant-time RSA-512 digital signature (PSS + SHA256) under 10 KB, testable on standard consumer laptops.

Specific Objectives:

1. Implement a branchless RSA-512 digital signature in C using CRT, Montgomery multiplication, and fixed-window exponentiation.
2. Ensure constant-time execution by removing secret-dependent branching and memory access patterns.
3. Collect timing traces and apply Welch's t-test to verify no correlation with secret key bits ($p > 0.05$).
4. Benchmark signing and verification performance.
5. Publish the complete implementation and documentation as open-source.

1.4 Motivation

The motivation for this research is the need for secure, accessible cryptographic implementations. Students and developers often lack simple, verifiable examples of constant-time algorithms. This project delivers a minimal, transparent RSA constant-time implementation that supports education, experimentation, and secure prototyping.

2 Related Works

Several researchers have studied how RSA implementations leak information through timing. Paul Kocher first showed that attackers can guess RSA private keys by measuring how long encryption or signing takes. After this, many studies introduced methods such as Montgomery multiplication, fixed-window exponentiation, and branchless algorithms to reduce timing variations during RSA operations.

Modern libraries like OpenSSL and BoringSSL use constant-time functions, but these implementations are large, complex, and difficult for beginners to understand. They also require advanced hardware optimizations, making them less suitable for learning and small research projects.

Other studies used statistical techniques such as Welch's t-test to detect timing leaks. This method checks whether execution time is related to secret key bits. If the p-value is greater than 0.05, it means the implementation does not show significant timing leakage.

Although many works focus on improving RSA security or testing for leaks, most existing constant-time implementations are not minimal or easy to study. There is very little work on small, simple, constant-time RSA-512 implementations designed for students or educational use. This research aims to fill that gap.

3 Research Gap

Most existing constant-time RSA implementations are very large, complex, or designed for special hardware. Because of this, they are difficult for undergraduate students to study or modify. There are very few small and easy-to-understand constant-time RSA implementations that students can use for learning or research.

In addition, only a limited number of studies have tested timing leaks in simple, minimal RSA programs. Very little work has been done on checking timing safety using easy-to-collect datasets on normal laptops. This research aims to fill this gap.

4 Materials and Methods

This research will be conducted using controlled timing experiments, statistical analysis, and secure coding principles.

Materials/Tools

- C compiler (to write and run RSA code)
- Computer with precise timing tools (to measure execution time of code)
- Python (for analyzing timing data using statistics and Welch's t-test)
- GitHub repository (to save and share code and results)

Procedures

1. Develop RSA-512 signing using constant-time Montgomery multiplication.
2. Implement fixed-window exponentiation and branchless CRT.
3. Conduct large-scale timing measurements.
4. Perform Welch's t-test on collected timing traces.
5. Benchmark performance against non-constant-time baselines.

Data Analysis

Timing data will be statistically analyzed to detect correlations between timing variations and key bits. A p-value greater than 0.05 indicates no significant timing leak.

5 Expected Outcome

The expected outcomes include:

1. A verified constant-time RSA-512 signature implementation under 10 KB.
2. Statistical evidence confirming resistance to timing side-channel attacks.
3. An open-source repository beneficial for students, educators, and researchers.
4. Contribution to secure software development and cryptographic education.

6 Ethical Approval

This research contains no human participation, personal data, or harmful activities. All experiments are conducted using self-generated RSA keys and timing datasets.

7 Gantt Chart

Work/Time (Months)	Nov-24	Dec-24	Jan-25	Feb-25	Mar-25	Apr-25	May-25	Jun-25
Title selection								
Proposal								
Literature Review								
Resource gathering								
Tools and Techniques								
Implementation								
Writing								
Publication / Conferences								
Presentation and Submission								